高价率运营 AI 工作台：约定驱动与 AI 编排的评测优化实践

原创 营销&交易技术 大淘宝技术 2026年7月16日 19:27 浙江 10人



本文介绍了“高价率运营 AI 工作台”中基于“约定驱动 + AI 编排”架构的自动化评测优化实践。针对 LLM Agent 场景下传统测试失效、反馈周期长等痛点，文章详细阐述了将评测体系作为核心工程产物的设计方案：通过标准化目录结构固化规范，利用 Coding Agent 实现自然语言驱动的全流程自动化；构建包含真实业务数据驱动的评测集生成、14 个通用维度与专项指标结合的 Rubric 体系、以及采用二元评分机制的 LLM Judge 双引擎架构（auto-evaluation 与 pinchbench-eval）。同时，文章深入剖析了金标设计中的信息隔离、粒度匹配等关键陷阱，并探讨了从半自动向全自动演进过程中，在评测入口、指标自动迭代及记忆机制等方面面临的技术挑战与应对策略，最终实现了将 Skill 可用性从主观判断转化为可量化、可复跑的工程闭环。

01 引言与背景

业务背景与建设目标

高价率运营是电商行业小二的日运营场景,涉及高价播报、数据分发、同款校验、价格跟进、归因、日报等长链路工作,频次高、协作链路长、规则复杂——单一行业横向每天耗时数小时,大促期间还要数倍叠加,且新人通常需要 2-3 个月才能熟练掌握全套操作。这是典型的"可标准化、可重复"工作,适合用 AI Agent 替代。

我们立项建设"高价率运营 AI 工作台",围绕目标管理、数据分发、价格跟进、日报产出 4 类可标准化事项,把人工操作沉淀为一组业务 Skill。但很快我们发现,真正的难点不在于"做出 Skill",而在于"持续保证 Skill 可用"——业务规则在迭代、上游数据接口在变、新场景层出不穷,任何一次 SKILL.md 的修改都可能让某个上线已久的 Case 突然崩掉。

为什么评测是工作台的核心问题

传统软件工程的"上线前测一下"在 LLM Agent 场景下基本失效:

1. 没有确定的输入输出:同样一句话,不同 session、不同上下文、不同模型版本,输出可能完全不同
2. 失败模式不可枚举:不像传统 bug 那样可以写 unit test,Agent 的"错"往往是"流程跳步""参数臆造""把内部字段直接抛给用户"这类语义错误
3. 改一处影响一片:SKILL.md 改一行约束,可能让原本通过的 10 个 Case 中有 3 个挂掉——而你不知道是哪 3 个

我们最早的版本是"开发自己跑跑、产品看一眼觉得对就上，评测比较宽泛评测分也很高",结果是线上反馈成为唯一信号:某个 Skill 跑了一周,小二抱怨"功能又不能用了", 这功能在上线时针对某些场景就存在的bug。并且这种反馈周期以"周"计,完全不可接受。

所以工作台从一开始就把评测体系作为一等公民来建设——把"Skill 是否可用"从主观判断,变成可量化、可复跑、可对比的工程产物。本文重点讲的就是这套评测体系:它的架构、它的金标设计、它踩过的坑、以及它如何反过来驱动整个工作台的演进。

工作台的整体设计

工作台采用"约定驱动 + AI 编排"的架构：通过标准化目录结构(`skills/` / `skill-data/` / `pinchbench-suite/`)固化规范,通过 Claude Code 作为编排器实现自然语言驱动的全流程自动化。整体形成三个层次:

> ● 业务层:16 个业务 Skill,提供 Agent 能力
> ● 规范层:14 个通用评测维度 + 各 Skill 专项 rubric + 216 条评测用例
> ● 执行层:auto-evaluation(评测大脑) + pinchbench-eval(执行引擎)双引擎

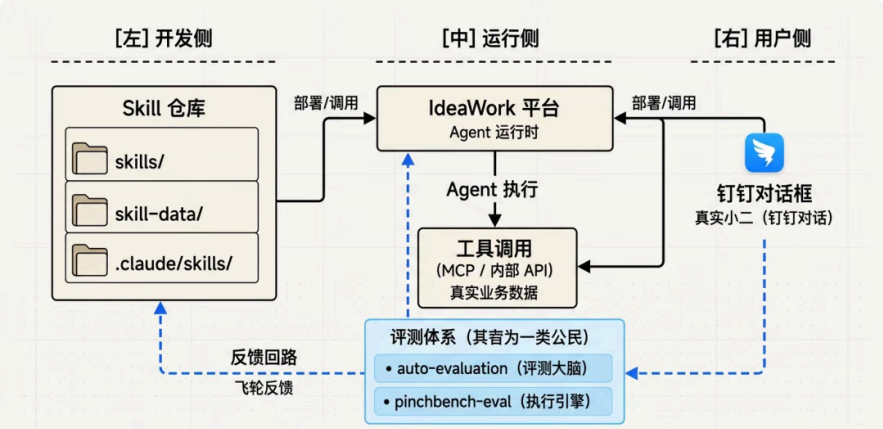


图 1：高价率运营 AI 工作台的整体结构。蓝色虚线展示了"评测体系作为反馈回路"的设计——评测不是开发后的一个步骤，而是把工作台连成闭环的关键基础设施。

后续章节按"架构 → 评测体系 → 落地效果 → 挑战与未来"展开,其中第三章详细拆解评测体系的每一个关键设计;但如果只想读最关键的部分,直接看 5.4 / 5.5 / 5.6 三节——那里讲的是评测飞轮跑通之后真正的三个硬骨头:入口(线上问题怎么进得了飞轮)、出口(修改建议如何真正改对地方)、记忆(每一次转动如何被记录下来)。这三个问题决定了飞轮是"转得动"还是"转得对"。

02 核心架构：约定驱动+AI编排

约定驱动:通过目录结构固化开发规范

工作台把"规范"沉淀到目录结构里,而不是写在另一份独立文档里——后续章节频繁出现的 `skills/<name>/SKILL.md`、`scripts/`、`constraint.md`、`rubric_config.json` 等路径,都来自这套约定。

```
skills/<skill-name>/
├── SKILL.md              # Skill 定义(YAML frontmatter + 执行流程描述)
├── scripts/              # 辅助脚本(Python/Shell)
├── references/           # 参考模板和示例(可选)
└── .claude/              # Skill 级别配置(可选)
```

SKILL.md 用 YAML frontmatter 强声明触发词等关键信息——后面 3.11 节"触发词重叠"、5.6 节"viewer 误触"两个坑都是建立在这个声明可被全局扫描的基础上:

```
---
```

```
name: seller-broadcast-sender
alias: 商家播报助手
description: 创建商家高价播报任务,推送到钉钉群或创建定时播报任务
triggers:
  - 商家播报
  - 商家高价播报
  - 商家比价播报
---
```

```
skill-data/                          # 源数据,纳入 Git
├── rubrics/                         # 14 个通用评测维度
├── rubric_config.json               # 通用指标配置
├── skills/<skill-name>/
│   ├── queries.json                 # 评测集
│   ├── rubric_config.json           # Skill 专用指标(可选)
│   ├── constraint.md                # 专用约束(可选)
│   └── examples.json                # 参考样例(可选)
└── scripts/sync_to_suite.py         # 同步到执行目录
pinchbench-suite/                    # 执行环境,不进 Git
├── queries.xlsx                     # 同步后的合并评测集
├── prompts/                         # 评测指标(rubric)
└── outputs/<run_name>/
    ├── runs/                        # Agent 执行结果
    ├── scores/llm_scores/human_scores/
    └── reports/                     # 汇总报告(MD + HTML)
```

为什么坚持这个分离？**3.5 节会展开讲**——`sync_to_suite.py` 这座桥让"评测源管理"和"评测执行"彻底解耦,改 queries 不会影响正在跑的评测,跑评测产生的中间文件也不会污染源数据目录。这条约定看似只是目录划分,实际上是后面双引擎架构能跑得稳的工程前提。

AI 编排:让目录约定真正"跑起来"

光有目录约定还不够——开发者每次创建 Skill 评测 都要手动 mkdir、写 frontmatter、配 rubric,**约定就会逐渐被绕过**(写少一两个文件、命名不规范、忘了同步)。我们的解法是把 Coding Agent 当作工作台的编排器,**让自然语言成为唯一的入口**,目录约定由编排器强制兑现。

一个具体例子:从一句话到一个完整 Skill 评测

用户输入:"评测商家播报 Skill"

Coding Agent 自动按目录约定创建:

- `skills/seller-broadcast-sender/SKILL.md`(含 frontmatter + 触发词 + 执行流程)
- `skills/seller-broadcast-sender/scripts/run_broadcast.py`(脚本骨架)
- `skill-data/skills/seller-broadcast-sender/constraint.md`(空约束模板)
- `skill-data/skills/seller-broadcast-sender/queries.json`(空评测集模板)

关键不是"AI 帮我建文件",关键是**这些文件名、字段、目录位置都不可协商**——开发者不再有机会"建在另一个地方""少建一个文件"。约定从规则变成了**默认且唯一的产物**。

编排器覆盖的四个环节

同一个机制贯穿整个产研闭环:

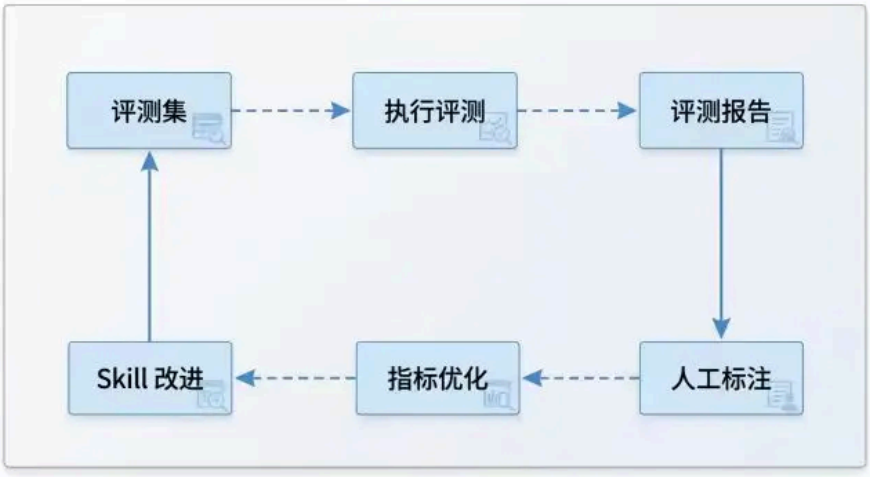
触发说法	AI 自动执行的事	涉及的目录约定
"新增需求"	建 Aone 需求 → 创 feature 分支	分支命名 `feature/<desc>-<date>`
"创建 Skill X"	见上面例子	`skills/` + `skill-data/skills/`
"评测 Skill X"	读 SKILL.md → 生成 queries → 同步 → 跑批 → 出报告	`skill-data/scripts/sync_to_suite.py` 桥
"保存修改 / 提交审批"	Git commit + 创 PR + 触发打包	`pinchbench-suite/outputs/` 不入 Git



自动化评测架构设计

评测架构的核心创新：把评测从"一次性快照"变成"持续联动"

整个第三章的设计可以浓缩成一句话:**评测集、评测指标、Skill 定义三者不是独立产物,而是同一个飞轮的三个齿轮**——任何一个动了,另两个都要跟着动,且动作能被自动化。



为什么"三者联动"是关键判断

我们最早不是这么设计的。v1 的评测集是测试同学独立写的、评分标准是产品同学口头约定的、Skill 改进靠开发同学看反馈——三者由不同角色维护,**结果就是评测集追不上 Skill 演进、评分标准每个人理解不一样、改完没人验**。这套用了不到一个月就停了。

现在的"三者联动"有一个隐含前提:**它们必须共享同一份语义来源**。我们的做法是把 SKILL.md 当作单一事实源——评测集由 LLM 读 SKILL.md 自动生成、评分指标在 SKILL.md 的约束章节里同步声明、Skill 改进的产物本身就是新版 SKILL.md。三者绑在同一份文件上,**自然就同步演进**。

> 至于这个飞轮**真正能不能转出价值**——5.4 / 5.5 / 5.6 给了诚实答案:中段已经稳了,但入口、出口、记忆三处还远没解。这一章讲的是"齿轮怎么咬合",第五章讲的是"咬合之后才发现真正的难点在别处"。建议两章对照着看。

评测集生成：基于真实业务数据

评测集的质量直接决定了评测的有效性。如果评测集偏离真实业务场景,即使评测分数再高,也无法保证线上效果。我们的评测集生成遵循"真实数据驱动"原则。

真实数据查询(MCP工具调用)

评测集不是凭空构造的,而是基于真实业务数据。AI会调用MCP工具查询三类数据:

- **员工岗位数据**:员工负责的行业、商家列表,用于构造"我的高价商品查询"等场景
- **高价率目标数据**:各行业的高价率目标及完成情况,用于构造"高价率目标查询"等场景
- **高价商品汇总数据**:岗位级高价商品统计,包括商品ID、商家名称、价格差异等

这些数据通过MCP工具实时查询,确保了评测数据的真实性和时效性。比如,评测"商家播报"Skill时,会使用真实的商家名称、比价池配置、钉钉webhook等,而不是虚构的测试数据。

AI场景分析与Case生成

有了真实数据后,AI会分析Skill定义,识别测试场景。评测用例分为六大类:

场景类型	说明	占比	示例
典型场景	参数完整或需逐个引导	40%	"帮我创建逸阳旗舰店的高价播报"
边界场景	参数异常、商家模糊、权限不足	30%	"帮我创建不存在的商家的播报"
追问场景	多轮对话参数补充	15%	用户先说"创建播报",再逐个补充参数
格式验证	输出格式是否符合规范	5%	验证表格、链接格式是否正确
触发词测试	关键词识别验证	5%	"商家播报"、"商家高价播报"等触发词
真实Case	用户提供的真实query	5%	来自线上日志的真实用户输入

AI会根据Skill定义中的execution_flow,为每个流程步骤至少生成1个Case。比如,"商家播报"Skill包含4个步骤:

1. 查询播报条件
2. 获取高价汇总
3. 生成消息
4. 推送钉钉群

AI会为每个步骤生成典型场景Case,同时考虑边界情况(如商家不存在、权限不足)和追问场景(如用户先说"创建播报"再补充参数)。

评测集字段协议设计

评测集采用结构化的JSON格式,包含以下字段:

字段	必填	说明	示例
`task_id`	是	唯一ID	`task_seller_broadcast_typical_001`
`category`	是	分类(典型/边界/追问/格式验证/触发词测试/真实Case)	`典型场景`
`evaluators`	是	评测器配置	`llm:task_seller_broadcast_sender`
`prompt`	是	用户输入	`帮我创建逸阳旗舰店的高价播报`
`reference_data`	否	参考数据(标准答案、检查点等)	`{"expected_flow": [...]}`
`followup`	否	动态多轮配置(JSON)	`{"max_turns": 20}`
`empId`	是	员工工号	`236704`
`new_session`	否	是否新建session	`true`

`reference_data` 字段是评测的关键,它定义了期望的执行流程、关键检查点、预期的输出格式等。比如:

```
{
  "expected_flow": [
    "查询播报条件",
    "获取高价汇总",
    "生成消息",
    "推送钉钉群"
  ],
  "checkpoints": [
    "商家名称必须原样传递",
    "必须调用run_broadcast.py",
    "创建态必须等待用户确认"
  ]
}
```

这些检查点会被LLM Judge用于评分,确保评测的准确性和一致性。

评测指标体系:通用指标与专项指标

评测指标(Rubric)是评分标准,它定义了从哪些维度评估Agent的能力、每个维度的权重、什么情况判0分等。我们的指标体系包含三个层次:

维度库的沉淀过程:不是设计出来的,是踩出来的

我们在 `skill-data/rubrics/` 目录下维护了 14 个通用评测维度。坦白讲,这 14 个**不是一开始规划好的**——最早只有 3 个(路由准确性、流程遵循、参数正确性),其他 11 个全是后来"不断优化后补进来"的。

举几个真实的来历:

- `name_code_distinction` 这个维度的诞生,是因为有次 Agent 把内部字段 code 直接显示给小二——之前 rubric 里根本没有"展示用名称、传参用编码"这条,加完之后才覆盖住 `tool_compliance` 是因为有 Skill 私自调了不在白名单里的内部 API
- `output_compliance` 是因为某个 Skill 在输出里带了 SQL 片段

这些维度共同的特点是:**事先想不到,出过一次事就必须有**。所以维度库的扩张节奏其实和 Skill 上线节奏强相关——每多上一批 Skill,大概率会冒出 1–2 个新维度。

4 类维度的分类逻辑

14 个维度按"约束什么"可以归成 4 类:

类别	关心什么	代表维度	是否常 severe_violation
流程类	Agent 是不是按设计的步骤在走	`route_accuracy_general` `flow_adherence` `task_selection`	✅ 大多数是
参数类	调工具时业务字段有没有传错	`param_correctness` `name_code_distinction`	✅ 大多数是
输出类	给用户看的东西合不合规、清不清晰	`output_format` `expression_clarity` `output_compliance`	❌ 一般不是
效率/合规类	资源消耗、调用合规、异常处理	`performance_efficiency` `response_time` `tool_compliance` `exception_handling`	部分是

剩下几个(如 `intent_understanding`、`route_accuracy_broadcast`)按场景归到对应类别。完整清单和每个维度的评分规则在 `skill-data/rubrics/`,这里不展开列——因为**列出来不会帮你做评测,理解"它们怎么来的"才会**。

举一个维度文件长什么样: `param_correctness`

每个维度都有独立的 Markdown 文件描述评分规则。

比如 `param_correctness.md` 定义:

```
# 参数正确性
## 评分规则
```

```
- **1.0分**:必需参数完整,业务字段完全照搬原文传递
- **0.0分**:必需参数缺失,或业务字段有任何修改
## 典型违规
- 商家名称加空格、改字、同义替换
- 传入了不存在的参数
- 参数值类型错误(如传入字符串而非数字)
```

通用指标配置(5个核心维度)

对于没有特殊要求的 Skill，可以使用通用指标配置(`skill-data/rubric_config.json`)评估器：

```
{
  "rubric_name": "task_sanity",
  "description": "通用评测指标(适用于无专用指标的Skill)",
  "score_range": [0, 1],
  "pass_threshold": 0.7,
  "dimensions": [
    {"file": "route_accuracy_general.md", "weight": 0.40},
    {"file": "intent_understanding.md", "weight": 0.10},
    {"file": "exception_handling.md", "weight": 0.10},
    {"file": "performance_efficiency.md", "weight": 0.30},
    {"file": "expression_clarity.md", "weight": 0.10}
  ]
}
```

这个配置包含5个维度,权重总和为1.0,通过阈值0.7。如果一个Skill没有特殊要求,直接使用这个通用配置即可。

专项指标定制(Skill专用约束)

对于有特殊要求的Skill,可以定制专项指标。比如,"商家播报"Skill有特殊约束:

- 商家名称必须原样传递(任何修改判0分)
- 必须等待用户确认(创建态跳过确认判0分)
- 只能使用Bash工具(使用MCP工具判0分)

这些约束在`skill-data/skills/seller-broadcast-sender/rubric_config.json`中定义:

```
{
```

```
  "rubric_name": "task_seller_broadcast_sender",
  "description": "商家播报助手专用评测指标",
  "dimensions": [
    {
      "file": "route_accuracy_general.md",
      "weight": 0.20,
      "judge_rule": "正确触发seller-broadcast-sender Skill,触发关键词:商家
    },
    {
      "file": "flow_adherence.md",
      "weight": 0.30,
      "severe_violation": true
    },
    {
      "file": "param_correctness.md",
      "weight": 0.30,
      "severe_violation": true
    },
    {
      "file": "performance_efficiency.md",
      "weight": 0.20
    }
  ],
  "severe_violation_rule": "若任一severe_violation维度为0.0,则整体评分上限为
}
```

这里的关键是 `severe_violation` 标记:如果一个严重违规维度得0分,整体评分不再按权重加权,而是取所有严重维度的最小值。这确保了核心能力的底线不被其他维度的高分稀释。

评测执行与评分

评测集和指标定义好后,进入执行阶段。评测执行包含三个关键步骤:执行Agent任务、LLM Judge评分、人工标注校验。

LLM Judge评分原理

Agent执行完成后,需要对其输出进行评分。我们采用LLM Judge机制,由Claude作为裁判,根据评测指标对输出打分。

评分流程:

1. 读取评测指标(rubric),解析出维度定义和权重
2. 读取Agent输出(runs/<task_id>.json)和reference_data

3. 对每个维度,LLM独立打分(0.0或1.0,禁止中间分数)
4. 计算加权平均分:final_score = Σ(weight_i × mean_i) / Σ(weight_i)
5. 处理严重违规:如果有severe_violation维度得0分,整体评分取严重维度的最小值

LLM Judge的关键在于"二元评分"框架:每个维度得分只能是0.0或1.0,禁止中间分数(如0.5)。这避免了LLM打分的模糊性,让评分更客观、可解释。

为什么是二元评分,而不是连续评分?

在初期方案中我们尝试过0–1连续评分(允许0.3、0.5、0.8等中间值),与最终采用的二元评分做了对照实验。在同一批30个用例上,由3个评测者(2人+1个LLM Judge)分别打分,结果如下:

评分方式	评测者间一致性(Cohen's κ)	复评稳定性(同一评测者两次打分一致率)	问题定位效率
连续评分(0-1)	0.42(中等一致)	68%	低,0.6和0.7的差异难以解释
三档评分(0/0.5/1)	0.61(较高一致)	81%	中,但0.5档争议大
二元评分(0/1)	**0.83(高度一致)**	**94%**	高,任何不通过都需明确指出原因

二元评分的本质是"强迫评测者做出判断":要么这个维度合格,要么不合格,中间地带不存在。这带来三个工程上的好处:

1. **可复现**:同一份输出,不同时间、不同评测者打分基本一致,评测结果可信
2. **可累计**:不通过的维度必须给出"违反了哪条规则",这些理由可以累积成新的checkpoint,反哺评测集
3. **可优化**:0/1的判断结果可以直接驱动决策(改Skill还是改指标),不需要再做"0.6到底算不算合格"的二次判断

代价是:对评分规则的描述要求更高——必须能够明确划线,不能模棱两可。这反过来促使我们把每个维度的评分规则写得越来越清晰,也是评测体系自我进化的一部分。

评分提示词示例:

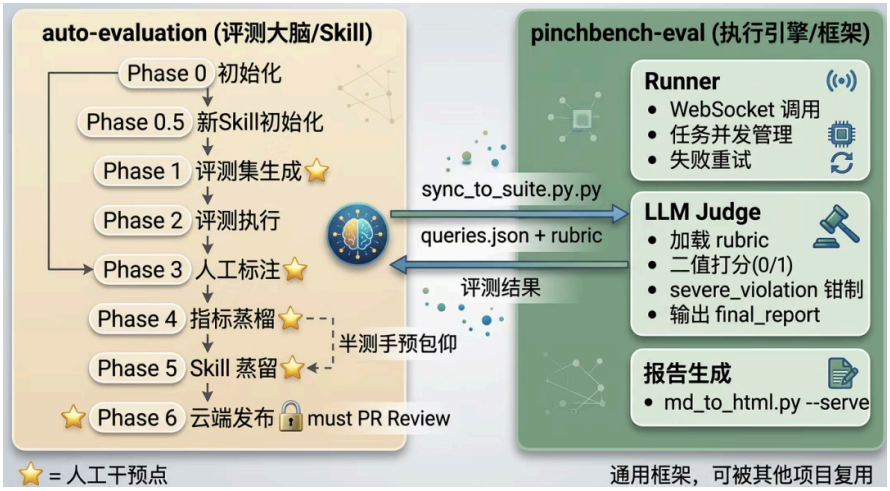
```
你是一个评测专家,请根据以下指标对Agent输出打分。
## 评测维度:参数正确性
- 1.0分:必需参数完整,业务字段完全照搬原文传递
- 0.0分:必需参数缺失,或业务字段有任何修改
## Agent输出
{agent_output}
## 参考数据
{reference_data}
```

```
9  ## 评分
10 请判断Agent输出是否符合参数正确性要求,只能给出0.0或1.0分,并说明理由。
```

双引擎架构：auto-evaluation 与 pinchbench-eval

前面讲的是"评测做什么",这一节讲"评测靠什么落地"。整个评测体系不是一个工具,而是两个分工明确的引擎:**auto-evaluation(评测大脑)** 负责生成、蒸馏、闭环;**pinchbench-eval(执行引擎)** 负责跑批、评分、报告。两者通过 `skill-data/` 这个共享数据层联动。

auto-evaluation:6 Phase 闭环的"评测大脑"



auto-evaluation 是一个 Coding Agent Skill,通过自然语言触发,把"评测怎么演进"这个流程拆成 6 个阶段:

Phase	职责	关键产物
Phase 0	评测环境初始化、Skill 选取	工作目录、配置
Phase 1	评测集生成(auto/user/mixed 三模式)	`skill-data/skills/<skill>/queries.json`
Phase 2	调用 pinchbench-eval 执行评测	跑批结果交给执行引擎
Phase 3	人工标注 + 误判/漏判分析	`human_scores/`
Phase 4	指标蒸馏(根据标注结果优化 rubric)	`rubric_config.json` 修订建议
Phase 5	Skill 蒸馏(根据评测结果优化 SKILL.md)	SKILL.md 修订建议
Phase 6	云端发布(打包 ZIP 上传 IdeaWork)	已发布的 Skill 版本

闭环的关键约束有三条:**最大 10 次迭代**(防止无限循环)、**问题识别率阈值 0.95**(评分一致性低于该值不允许进入下一阶段)、**AI 不能自动应用修改**(所有 SKILL.md / rubric 修订都必须人工 Review 后落盘)。这三条约束保证了"自动化不等于失控"。

pinchbench-eval:可整包迁移的执行引擎

pinchbench-eval 是从评测大脑里抽离出来的纯执行引擎,目前已经迭代到 v2.29.0,核心特点是 **项目无关**——它不依赖高价率业务,可以整包拷贝到任何使用 Skill 体系的 Agent 项目复用。它的设计目标是把"跑评测"这件事工程化,而不是写在某个一次性脚本里。

执行引擎的职责边界很清晰:**只负责"按 queries.xlsx 跑、按 rubric 评分、按 13 章骨架出报告"**,不负责"该跑什么、该改什么"——后者是 auto-evaluation 的职责。这种分工让两个引擎都能独立演进:auto-evaluation 关注流程编排,pinchbench-eval 关注执行性能。

两层之间的同步桥

`skill-data/` 是两个引擎的共享数据层(纳入 Git),`pinchbench-suite/` 是 pinchbench-eval 的运行时目录(不纳入 Git)。中间的同步靠 `skill-data/scripts/sync_to_suite.py`:

- 把分散在 `skill-data/skills/<skill>/queries.json` 的评测集合并成 `pinchbench-suite/queries.xlsx`
- 把各 Skill 的 `rubric_config.json` 组装成 `pinchbench-suite/prompts/task_<skill>.md`
- 把通用维度从 `rubrics/` 拷到执行目录,供 LLM Judge 使用

这个同步桥让"评测数据的源管理"和"评测的执行环境"彻底解耦——开发同学在 `skill-data/` 里改 queries 和 rubric,不会影响正在执行的评测;评测执行产生的 outputs 也不会污染源数据目录。

执行引擎已当前提供 6 种运行模式(全量/单Case/断点续跑/标注/浏览/编辑),支持并发流水线、ETA 心跳和 HTML分享。这些都是让评测能从'开发自己跑跑'变成'团队都能用'的工程基建。

评测报告与对比分析

评测完成后生成 13 章 Markdown 报告(经 `md_to_html.py --serve` 转自包含 HTML 可分享),按"快速看整体 / 定位具体问题 / 诊断评分系统本身 / 支撑优化决策 / 成本与对照"分区,让不同角色直奔自己要看的那块。

整套报告设计上**最重要的一节是 五 "Auto 评分和 LLM 评分差距 ≥0.2 的 Case单独列出"**——这是评测系统的**自检通道**:Auto 是规则评分,LLM 是语义评分,两者大幅分歧的 Case几乎都对应"评测标准本身有歧义",**是 rubric迭代最值钱的输入**。其他章节是给不同角色看分数,这一章是评测系统在反过来检查自己。

报告还会自动做新旧版本对比(总分差 + 各维度趋势)和低分根因聚类(路由错误 / 流程跳步 / 参数错误 /性能低),为 Phase 4/5 的指标蒸馏 / Skill蒸馏提供原料。具体表头和样例不展开——它们是工具产物,不是文章要讲的设计。

评测集的"金标"设计

评测集的灵魂是 `reference_data` 字段——它定义了"什么是对的"。这个字段怎么设计,直接决定了 LLM Judge 能不能给出靠谱的评分。

reference_data 的真实结构

我们在实践中沉淀出一组半结构化的字段约定(没有强 Schema,各 Skill 在 `constraint.md` 里自行约定),抽样几个真实 Skill 的 `reference_data` 示例:

```
// query-high-price-rate (商品维度查询)
{
  "param_type": "item_id",
  "expected_flow": ["岗位校验","参数识别","获取比价任务","用户选择任务","执行商
  "expected_task_count": 2,
  "expected_output": "item_format",
  "canQueryTasks": ["2024_国补..._竞对全平台同店", "1972_国补..."],
  "followup_expectation": "Agent 展示多个比价任务让用户选择..."
}
// follow-price-progress (跟价进展)
{
  "expected_flow": ["岗位校验","参数解析","调用跟价进展接口"],
  "expected_parameter": {"sellerName": "逸阳旗舰店"},
  "expected_output": {
    "sections": ["进展概览","明细列表"],
    "format_requirements": ["必须用表格","商家名展示中文"]
  }
}
```

可以看到几个共性字段:`expected_flow`(期望执行步骤)、`expected_<param>`(关键参数)、`expected_output`(输出结构)、`canQueryTasks` / `expected_task_count`(可枚举的合法选项)。这些字段都是**给 LLM Judge 读的**,不是给代码做 assertion 的。

金标设计的踩坑教训

`reference_data` 这个字段我们前后改了 5 个版本,每一版的修改都源于一次"评测看起来对、但其实不对"的事故。这里挑 4 个最值得说的坑。

坑一：被测 Agent 看到了金标——已完成隔离

最早的坑是**信息泄漏**:`reference_data` 一开始是直接拼在被测 Agent 调用上下文的同一份 prompt 里的(那时候 evaluator 和 runner 没拆干净)。结果 review 评测结果时发现一个匪夷所思的现象——某个 Skill 在所有 Case 上都给出了"和 reference_data 完全一致"的输出,**包括 Agent 不可能知道的内部商家 ID**。

复盘后才意识到:Agent 的对话上下文里**意外混进了金标**,等于"开卷考试"。这一版评测的所有分数全部作废。

应对(已完成):把 `reference_data` 在执行链路里**严格隔离到 LLM Judge 一侧**——

- pinchbench-eval 的 runner 调用 Agent 时,**只传 `prompt` 字段**,`reference_data` 完全不进入被测 Agent 的上下文
- LLM Judge 在评分阶段才从 `runs/<task_id>.json` 加载 `reference_data`,且只用于评分推理
- 在 `evaluator.py` 里把"runner 输入"和"judge 输入"做成两个独立的 dataclass,**类型上就不允许混用**——这一步是真正解决问题的关键,文字提醒永远是兜不住的

这个隔离做完之后,"自评成功"这一类幻觉评分就彻底消失了。**这条经验我们最大的收获是:LLM 系统里的"信息隔离"必须做到类型层面,不能靠 prompt 里写"请不要看 X"**——LLM 不会真的不看。

坑二：`expected_flow` 列得太细,导致"流程对了但分数低"

第二版评测我们对 `expected_flow` 写得过细,把每个内部函数调用都列了进去,比如:`["参数校验", "查岗位权限", "拉取商家配置", "查比价池", "调价格 API", "格式化输出"]`——总共 6 步。

结果有大量 Case 的 `flow_adherence` 维度被判 0 分。深入看才发现,Agent 实际上**走对了流程**,但它把"参数校验"和"查岗位权限"合并成一次工具调用,导致 LLM Judge 数下来"少了一步",直接判违反流程。

应对:粗化 `expected_flow`,只保留"语义大阶段"

把预期步骤从 6 步合并成 4 个语义大阶段:

- **原细粒度**:参数校验 → 查岗位权限 → 拉取商家配置 → 查比价池 → 调价格 API → 格式化输出
- **粗粒度**:前置校验(含参数 + 权限)→ 拉取配置与比价数据 → 调用价格接口 → 格式化输出

LLM Judge 只校验大阶段顺序,不卡死内部是否合并。同时在 rubric里加一句"允许步骤合并,只要业务语义等价"。这条规则现在写进了 `flow_adherence.md` 的评分规则里——**金标的粒度匹配,比金标的完整性更重要**。

坑三:`reference_data` 用 LLM 自动生成,被"答案污染"了

为了加快 Case 入库,我们试过让 LLM 直接根据 Skill 定义自动生成 `reference_data`。看起来很省事——直到一次评测后做人工抽查,发现 LLM 生成的 `expected_output` 和 Agent 实际输出**几乎一模一样**,包括同样的错别字、同样的格式 bug。

原因不复杂:LLM 在生成金标时,**用的就是同一个 Agent 跑出来的输出当参考**。等于"让 Agent 自己出题、自己批卷"。这一类 Case 在评测里无论 Agent 怎么跑都得满分,**评测信号完全失效**。

应对:LLM 自动生成的 `reference_data` 必须**人工过一遍才能入库**(已经在 5.1 节"当前状态"里提过)。同时给 Phase 1 加了一条硬规则:**生成金标时,LLM 不能引用任何已存在的 Agent 实际输出作为参考**——只能基于 SKILL.md 的定义和真实业务数据推导。

坑四:多轮 Case 的金标只覆盖了"最后一轮"

追问场景的 Case 是多轮对话——用户问 → Agent 答 → 用户追问 → Agent 再答,可能持续 5–10轮。早期版本我们的 `reference_data` 只描述了"**最后一轮 Agent 应该输出什么**",对中间轮次没约束。

结果出现一类很隐蔽的问题:**Agent在中间轮次反复无效追问、绕圈子、甚至偏离用户原始意图,但只要最后一轮把答案凑对了,LLM Judge就判通过**。典型表现:用户第 1 轮已经给了商家名,Agent 在第 2–4 轮反复追问"请问商家是?",到第 5轮才回到正轨给出结果——线上小二早就放弃了,但评测分数满分。

应对:`reference_data` 增加 `followup_expectation` 字段,描述"**每一轮**应该保持哪些上下文、不应该出现哪些无效追问、轮次上限是多少"。同时把 LLM Judge的 prompt 改成**逐轮独立打分,再聚合**——任何一轮出现"重复追问已知信息"或"偏离原始意图",该 Case 整体降级,而不是只看最后一轮的输出。

4 个坑的共同点

回头看,这 4 个坑——信息泄漏、粒度过细、答案污染、多轮覆盖——**没有一个是"金标内容写错了",全是"金标的工程结构没设计对"**。金标的内容可以靠业务理解打磨,但金标的**结构问题**(谁能看到、什么粒度、谁来生成、覆盖到哪)只能靠真实事故才暴露。

这也是为什么我们在 SKILL.md 里把"reference_data 设计规范"独立成一节——**比起教别人怎么写金标,更重要的是告诉别人金标系统设计上必须躲开哪些坑**。

LLM Judge 评分 prompt 的真实结构

LLM Judge 不是黑盒——它的输入(prompt)、输出(score JSON)、调用方式都可以拆开看。这一节剖析真实的 prompt 结构。

prompt 6 段式结构

`pinchbench-suite/prompts/task_<skill>.md` 是 LLM Judge 的入口 prompt,采用 6 段式结构:

```
1. YAML frontmatter
   - rubric_name、score_range: [0, 1]、pass_threshold: 0.7
   - dimensions 列表(name/label/weight/description/severe_violation)
   ⚠️ frontmatter 在喂给 LLM 之前会被剥掉,只供人工标注页 schema 使用
2. 维度逐一展开(每个 rubric 维度独立 §小节)
   - 1.0 / 0.0 的判定条件
   - 通用检查点表
   - 典型违规示例表
   - Skill 专属判定规则
3. 评估输入占位符
   - {user_instruction}、{assistant_output}、{reference_data}
   - 渲染时由 _build_variables 替换
4. 输出 schema(JSON)
   - {"scores": {<每个维度>: 0.0|1.0}, "mean": <加权均值>,
      "reasoning": ..., "token_usage": {...}}
5. 计算规则(显式公式)
   - mean = route_accuracy*0.15 + param_correctness*0.2 + ...
   - severe_violation 钳制规则:若任一严重维度 0.0,
     则 mean 取严重维度最小值,不按权重计算
6. 框架接入说明
   - 用 Write 工具保存 JSON 到 {{OUTPUT_PATH}}
```

第 6 段是关键的工程设计:**让 LLM 自己用 Write 工具把评分写到文件**,而不是返回字符串等框架解析。这避免了"LLM 输出多了一段废话导致 JSON 解析失败"的常见坑——文件写出来就是文件,框架直接读。

LLM Judge 漂移防护——目前几乎没有

直白讲:**LLM Judge 对自己的评分一致性,目前没有专门的代码层防护**。

- 没看到固定 seed
- 没看到固定 temperature(`config.json` 只配置了超时和重试)
- `retry.judge` 是失败重试,不是分数差异时复评

- 唯一的兜底是 Phase 4 的人工标注 + REPORT的"人工 vs LLM 对比"

为什么不做技术防护？因为 LLM Judge 的漂移不是"今天 0.7、明天 0.5"这种数值漂移,而是"今天觉得这个算流程跳步、明天觉得不算"这种语义漂移——固定 seed/temperature 解决不了语义层面的不稳定。**真正的解法是把 rubric 写得越清晰、判定边界越明确,漂移空间就越小**。这是为什么我们花大量精力写 rubric 文档而不是去研究采样参数。

承认局限的一面:这意味着评测分数有一定波动是正常的。我们在实际使用时,**不会因为单次评测分数波动 ±0.05 就下结论**——必须看趋势,或者跑 2–3 次取均值。这是工程实践,不是技术问题。

评测真实踩过的坑

工作台不是一蹴而就的,评测体系尤其踩过不少坑。这一节列出真实的踩坑记录,供参考。

坑 1:LLM 不会按字面"自律"——写文件会半截、读约束会挑软的

两个看起来不相关的现象,其实是同一类问题。

现象 A:LLM Judge 写评分文件偶发失败。0504 评测对比报告 18 注解记录:基线评测有 7 个 Case 的 `final_score` 为 None,0504 评测有 1 个,根因是 LLM 用 Write 工具落盘 score JSON时偶发写半截或写错路径。后续在 reporter 里加了"final_score 为 None 时降级标记"逻辑,**但写入失败本身没有彻底根治,只是变得"可观测"了**。

现象 B:SKILL.md 写"建议 X"和"必须 X",执行强度差很多。同样在 0504 评测里,SKILL.md 写"建议先调用 X 再调用 Y"时,Agent 经常跳过 X 直接 Y;改成"**【强制】必须先调用 X**"后,Agent 才会乖乖按顺序。这听起来反直觉(都是文字描述,为什么强度有差),但实测就是这样。后来在所有关键步骤前加了 `【强制】` 或 `【禁止】` 标记,并配合 rubric 的 `severe_violation` 机制——双管齐下才把流程跳步控制住。

两个现象的共同根源:LLM 不会真的按你 prompt里写的"请写完整""建议这样做"自律——它会写半截、会挑软的看。**任何"靠 LLM自觉"的设计都要在外面套一层硬约束**:Write 失败要在 reporter 层兜底降级,prompt 软描述要靠强语气 + severe_violation 钳制兜底。

坑 2:同步时机错误,数据被覆盖

`phase2-eval.md` 用一整节强调:"禁止提前同步:Phase 0.5/1/4 时不同步到 pinchbench-suite,只有用户明确说'开始评测'才同步"。

这种针对性"禁止"措辞通常意味着——**曾经发生过过早同步导致 `pinchbench-suite/queries.xlsx` 被覆盖**。当时是 Phase 1 改完 queries.json 立刻 sync,把上一次评测正在用的

xlsx 冲掉了,导致评测结果对不上。后来加了"必须用户明确说'开始评测'才执行 sync_to_suite.py"的硬约束。

坑 3:AI 自创非标准字段

`phase4-distill.md` 明确点名禁止使用 `skill_specific_checks` 这种"非标准字段"。这种"具名禁止"几乎都是反推——某次蒸馏时 AI 自己造了一个新字段,后续 sync 脚本不识别,评测就跑不动了。

教训:**LLM 倾向于"创造性地"扩展 schema**。任何"自由扩展"都要在 schema 校验时挡住,而不是寄希望于 LLM 自律。

坑 4:Touch 词重叠导致路由误触

0504 评测倒退 −0.1007 分,34 个用例从满分降到 0,主要根因之一是 **触发词边界模糊**——`price360-item-history-compare`(商品价格历史对比)和 `query-high-price-rate`(高价率多维度查询)的触发词有重叠,某些 Case 被错误路由到了另一个 Skill。

修复:重新梳理价格相关 Skill 的触发词,确保互斥。这个坑直接催生了第 3.3 节里的 `route_accuracy` 维度的强化——现在每个 Skill 的 `rubric_config.json` 里都会显式列出"本 Skill 的触发关键词"和"易混淆的兄弟 Skill"。

这些坑的共性

回头看这 4 个坑,有两个共同模式:

1. **LLM 不会按字面自律**:坑 1(写文件半截 + 忽视非强制语气)是最典型的——你 prompt里写"请写完整""建议先做 X",LLM 会写半截、会挑软的看。应对是**在 LLM 之外套一层硬约束**:Write 失败靠 reporter 兜底降级,软描述靠 `【强制】` 标记 + `severe_violation` 钳制兜底
2. **触发条件不能只看关键词**:坑 2(同步时机)、坑 4(viewer 误触)都是"关键词命中但上下文不对",坑3(触发词重叠)则是另一面——"关键词本身就不该唯一命中"。应对是**触发条件加状态判断和互斥校验**:不仅看说了什么,还看处在什么阶段、是否和兄弟 Skill 冲突

这些经验已经沉淀到 auto-evaluation 里。新人接手只要老老实实跟着走,基本不会再踩同样的坑——但这恰好说明,**评测体系的成熟度,体现在它"曾经踩过多少坑"上**。

04 业务落地效果:从约束到优化

Skill约束体系的建设成果

从 2026 年 4 月中旬启动至今,经过约半个月的高强度迭代,我们建立了一套完整的 Skill 约束体系,包括 16 个业务 Skill 的规范化、14 个通用评测维度的沉淀、以及各 Skill 专项指标的制定。

每个 Skill 都遵循标准化目录结构(SKILL.md + scripts/ + constraint.md + rubric_config.json)。**15 个 Skill 定制了专项 rubric_config.json,1 个走通用兜底配置**——这个比例本身是个有用信号:说明绝大多数业务 Skill 都有"通用维度兜不住"的专项约束,这也是为什么 3.3 节强调"通用 + 专项"两层指标的必要性。

完整 Skill 列表见代码仓库 `skills/` 目录。

沉淀这件事真正的价值,不是"我们沉淀了 14 个",而是"我们有了一个稳定的扩展机制"——下次再遇到新场景,知道往哪一类下面加,知道怎么写评分规则、怎么跟历史 rubric 兼容。这个机制本身比清单更值钱。

各Skill专项指标的制定

除了通用维度,每个Skill还有自己的专项指标,用于约束特定的业务规则。比如:

- **seller-broadcast-sender**:商家名称必须原样传递(任何修改判0分)
- **query-high-price-employee-top**:必须先调用getAllCompletionInfo获取任务列表,不能跳步
- **dingtalk-group-messenger**:必须先确认机器人类型,一次只问一个问题
- **price360-item-history-compare**:展示用Name(如"当前价"),传参用Code(如"curPrice")

这些专项指标在 `skill-data/skills/<skill>/constraint.md` 中明确定义,作为LLM Judge评分的依据。

评测数据集的建设历程

截至 2026 年 5 月 7 日,16 个 Skill 累计 216 条评测用例(平均 13.5 条/Skill)。**用例数差异不是建设进度差异,而是场景复杂度差异**——知识问答场景发散、播报场景流程固定,这是合理形态。

但比规模更值得说的,是建设过程中**真正吃力的部分发生了迁移**:

- **前 3 轮(从 0 到 ~150 条)**:重点在补 Skill 覆盖度和场景类型——典型 → 边界 → 追问 → 触发词,把"该测的事"列全
- **第 4 轮起(150 → 216 条)**:扩充明显放缓,重点转向**单条 Case 的金标质量**(reference_data 怎么写、checkpoint 怎么定、followup 怎么覆盖多轮)

这个转折点本身是个有用的信号:**评测集的瓶颈从"数量"变成了"单条质量"**。前 3 轮多新增 50 条 Case,可能能多发现 5 类问题;第 4 轮起多新增 5 条 Case,如果金标到位,能发现的问题反而比前者还多。3.8 节专门讲金标设计的 4 个真实坑,就是因为这一阶段我们绝大多数精力都花在了金标上,**而不是凑用例数**。

用例数本身没有"达标"概念——4 条够用还是 31 条够用,只能用"上线后还有没有意外"来回答,而不是事先拍数字。这一点和 3.2 节"评测集要从真实数据出发"是同一个思路:**评测集的好坏不在规模指标,而在它能不能拦下真问题**。

评测报告驱动Skill优化案例

三个案例,两个走完了"低分→标注→优化→复评通过"的标准流程,一个反复了两轮才稳住——后者比前两个更值得讲。

案例 1:商家播报助手 —— 用 severe_violation 收口流程违规

第一次评测整体未过阈,主要扣分集中在 `flow_adherence` 和 `param_correctness`。两个典型 Case:多商家场景下未展示选项、直接默认选第一个;创建态下未等用户回复"确认"就执行播报。

人工标注后 LLM Judge 的判定全部成立——确实是流程违规,而不是评测误判。处理动作有两步:`flow_adherence` 维度标 `severe_violation`(任一违反此维度的 Case 整体分钳到 0),SKILL.md 把"多商家展示选项"和"创建态等待确认"两条写成强制 Step。第二次评测两个维度均越阈,整体进入稳定。

这个案例的价值:`severe_violation` 不是为了惩罚分数,是为了让"红线类问题"不会被其他高分维度稀释。

案例 2:高价小二排名 —— 用参数白名单堵参数臆造

第一次评测出现两类严重违规:跳过 Step2/Step3 直接调底层接口、传入不存在的参数。前者是流程跳步,后者是模型把训练数据里见过的参数名"想象"成了真实参数。

人工标注同时指向了一条**新指标**——"参数白名单校验"(rubric 此前未覆盖参数臆造场景)。处理动作:SKILL.md 加显式参数表 + 脚本入口加 `ALLOWED_PARAMS` 校验,违反直接抛错。复评后参数臆造归零、流程跳步通过 SKILL.md 强化收敛。

这个案例的价值:Phase 3 的人工标注**不只是给已有 Case 打 verdict,还会反向产出"评测维度本身漏了什么"**——这是 Phase 4(指标蒸馏)的主要输入来源。

复盘启示

这个案例暴露了三个问题:

1. **指标之间存在张力**:把任一指标推到极致都可能伤害其他指标,优化要看整体而非单点

2. **绝对约束需要慎用**:"绝对禁止X"在边界场景往往会带来反效果,优先用"分场景规则"
3. **反向Case的价值**:每加一条约束,最好同时补充验证"约束不被滥用"的反向Case,否则容易出现这种"按下葫芦浮起瓢"的问题

后来我们把这三条总结成了内部评审清单,所有Skill优化PR必须自答这三个问题才能合并。

评测体系对工作台的真正贡献

回过头看,评测体系对工作台的贡献,**是把"Skill 是否可用"从主观判断变成了可复跑、可对比的工程产物**——但仅限于"线下、当前评测集所覆盖的范围内"。把这条边界先画清楚,再讲三层贡献。

1. 数据层面:沉淀了一套可复跑的基线

资产类型	当前规模	工作台中的位置
业务 Skill	16 个	`skills/`
通用评测维度	14 个	`skill-data/rubrics/`
评测用例	216 条	`skill-data/skills/<skill>/queries.json`
专项 Rubric	15 个	`skill-data/skills/<skill>/rubric_config.json`
历史评测 Run	3 次全量	`pinchbench-suite/outputs/`

任何 Skill 改动都能立刻跑一次评测、与基线对比,而不是"上线观察一周看反馈"。**但**:这套基线只反映"我们想到的场景",**线上真实问题怎么进来仍然靠人工每周扫群**——5.4 节会专门讲这个入口缺口。

2. 机制层面:把评测做成了一等公民

- `CLAUDE.md` 把"评测/评测集/评测指标"关键词强绑到 auto-evaluation Skill,自然语言即可触发
- `.claude/evaluation.md` 按需加载,评测规范不占用主上下文 token
- pinchbench-eval 抽象为项目无关的框架(v2.29.0),可整包迁移到其他 Agent 项目
- 形成"开发 → 评测 → 标注 → 蒸馏 → 上传云端"的端到端闭环

工具可以替换,但"评测是 Skill 上线前的必经环节"这个共识一旦形成,就成了工程文化。**但**:这个闭环里 Phase 4/5(指标蒸馏 / Skill 蒸馏)目前仍是"AI 建议 + 人工应用",建议采纳率还不

到 20%——5.5 节会拆这个出口的瓶颈。

3. 飞轮层面:让 Skill 沿着客观信号迭代

评测发现问题 → auto-evaluation 给修订建议 → 人工 Review 落盘 → 重新评测验证。这个循环把"Skill 改进"从"凭感觉""凭线上反馈"变成"凭评测信号",反馈周期从周缩短到小时。**但**:这个循环本身**没有记忆**——每一次评测都从零开始看现状,不知道上一次为什么改、改完到底解决了没有。5.6 节会讲这个记忆缺口,以及它为什么是 Phase 4/5 始终自动不起来的根因。

第五章接下来要讲的,正是上面三处加粗的"但"——飞轮的入口、出口、记忆三个缺口。这三处是接下来一年最值得投入的方向,也是这套体系距离"真正的全自动"还差的部分。

05 自动优化飞轮的技术挑战与解决方案

从全人工到半自动的演进

回头看这段演进,**最该说的不是"我们做对了什么",而是"我们走了哪些弯路"**。评测自动化听起来是个工程问题,实际上每一步自动化都伴随着一次"我以为可以,结果不行"的退步。

之前可以说基本都是依赖人工标注，第三版(现在):auto-evaluation + LLM Judge。这一版核心做了两件事:

1. **评测指标&评测集生成自动化**——基于真实 empId + Skill 定义,LLM 自动生成 6 类 Case(对应 3.2 节的典型/边界/追问/格式/触发词/真实 Case)
2. 评分自动化——LLM Judge 按 rubric 逐维度二值打分,替代人工对答案

这两件事让单次评测从"5 小时人肉"变成"40 分钟无人值守"。但我们也付了几笔代价,**这些代价当时没预料到**:

代价	具体表现	现在的状态
LLM 评分会漂移	同一份 Case 同一份 rubric,跑 3 次会有 5-10% 的分数差异	关键决策必须 N=3 取均值,见 3.10 节
评测集脱离真实问法	LLM 生成的问题太"规范",真实小二的口语化、打断、重启都没覆盖到	没解,见 5.4 节
评分依赖 rubric 质量	rubric 写得粗,LLM 就乱打;rubric 写得细,维护成本爆炸	14 个维度文件目前还能维护,再多就要拆分了
"自动"反而隐藏了问题	以前人工对答案能感受到"这个 case 怪怪的",现在 LLM 直接给个分数,**直觉缺失了**	强制要求人工抽查 10% 的 Case,见下文

当前状态:不是"全自动",是"自动+人工兜底"

诚实地说,现在的"半自动"距离真正的全自动还有很长一段距离。具体保留人工的点:

- **评测集入库前**:LLM 自动生成的 Case,**必须人工过一遍**才能进 queries.json——主要看 reference_data 是否合理、问题是否口语化得过头(LLM 偶尔会编出根本不可能出现的提问)
- **评分结果出来后**:每次评测结果**必须人工抽查 10% 的 Case**,主要看 LLM Judge 有没有"明显误判"——这个比例不是拍脑袋定的,是早期我们发现 100% 信任 LLM Judge 后,有一次连续两轮评测都"全绿"但线上同时段在抱怨,复盘发现是 LLM Judge 把一类边界 Case 全判错了
- **核心 Skill 上线前**:评测分数过线只是必要条件,**还要人工跑 3–5 个真实 empId 的端到端体验**才允许上线

这些"人工兜底"是教训换来的——每一条背后都对应一次"全自动信任出问题"的事故。

从这段演进里学到的一件事

如果让我们重做一遍,**最大的改变会是"更早引入 LLM Judge"**。我们在第二版人工对答案上耗了大约一个月,而 LLM Judge 的初版从写到能跑只用了 3 天——只是当时不敢相信"机器评分能比人靠谱"。**真正稀缺的不是 LLM 能力,而是把 LLM 嵌进现有工程流程的勇气**。

但反过来,**最大的不变会是"始终保留人工兜底"**。任何鼓吹"评测全自动、人完全不参与"的方案,目前在我们这个场景都不成立——LLM 评分的随机性、对边界 Case 的误判、对新场景的盲区,都需要人工作为最后一道防线。这一点 5.2 节会展开讲。

从半自动到全自动的挑战:指标自动迭代为什么这么难

5.4 和 5.5 讲的是飞轮的"入口"和"出口"。本节聚焦飞轮里另一个被低估、但同样卡住"全自动"的环节:**指标(rubric)本身的自动迭代**。

简单说就是:既然 LLM 能改 Skill、能给修改建议,**那为什么不能让 LLM 自己改打分标准呢?** 答案是——能,但**改完之后你完全没法判断它是改好了还是改坏了**。这是一个比 Skill 优化还棘手的问题。

为什么指标自动迭代特别危险:它是"测量自己"

Skill 改坏了,**评测分数会下降**——这是一个外部信号,可以纠错。 Rubric 改坏了,**评测分数也会变化**——但你不知道是 Skill 真的好了/坏了,还是单纯因为打分尺子变了。

打个比方:Skill 是"学生答题",rubric 是"判卷标准"。

- 学生答题改了,卷面分变了 → 你能看出学生进步了没

- 判卷标准改了,卷面分变了 → 你完全分不清是学生进步了,还是老师变松了

而我们要做的是**让 LLM 同时改学生答题和判卷标准**。这个组合一旦自动化,**飞轮失去了所有外部锚点**。这是指标自动迭代真正的困难——不是技术难,而是**没有可信的成功信号**。

踩过的坑:LLM 改 rubric 的三种典型失败

我们试过让 auto-evaluation Phase 4(指标蒸馏)自动给 rubric 改建议,跑了大约 5 轮后停了。失败的形式很有规律:

失败类型	具体表现	为什么会发生
标准放宽	LLM 把"必须包含 X"改成"建议包含 X",评分立刻全过	LLM 看到"低分 Case 太多",最简单的"修复"就是放宽标准
维度合并	LLM 把两个细维度合成一个粗维度,评测覆盖度下降	LLM 优化路径上的局部最优——"维度太多记不住,合并一下"
特例化	LLM 把通用 rubric 改成"如果 Case 是 X 类型,就用规则 Y",规则越来越多	LLM 在 prompt 里能看到具体 Case,会本能地针对 Case 调整

第三种最隐蔽——改完之后,rubric 在测试集上分数确实涨了,但**它已经退化成"针对测试集量身定制的判分逻辑"**,迁移到新 Case 上完全不行。这种过拟合连 LLM 自己都发现不了,因为它能看到的就是手头这批 Case。

我们当前的应对(都不完美)

- **rubric 改动只允许"加严"不允许"放宽"**:这是个粗糙规则,但能挡住"标准放宽"这一类失败。代价是 rubric 单调演化,无法应对"原 rubric 本身就过严"的情况
- **rubric 改动后必须用历史 Case 回归**:每次改 rubric,要在过去 3 次评测的所有 Case 上重跑,**评分排序应该基本不变**(具体地:历史上"明显高分"的 Case 不能跌成低分,反之亦然)。这是一种排序稳定性约束,不要求绝对分一致,只要求相对排序稳
- **rubric 修改 PR 必须人工 Review:且 Reviewer 必须不是修改建议的提出者**。这个规则很反直觉——为什么不能自己 Review?因为 Phase 4 的输出是 LLM 给的,如果同一个人既负责"采纳建议"又负责"Review",**他会倾向于相信 LLM 的判断**(这是真实发生过的)。强制换人 Review 是为了引入"独立怀疑"
- **rubric 版本号 + diff 留存**:每次 rubric 变更都打 tag,带上"为什么改、改完哪些 Case 受影响"的说明。半年后回头看哪个版本最稳,需要这些证据

仍然没解的硬骨头

诚实列一下:

1. **没有"独立 ground truth"**:rubric 是评测的判分标准,但 rubric 自己改得对不对,**没有更上一层的判分标准**——这是个递归问题。理论解是"用一份永不修改的人工标注金标集"作为

rubric 的对照,但维护这样一份金标集本身就是巨大的工程负担,我们目前没做

2. **rubric 改动和 Skill 改动如何"解耦"评测**:理想情况下应该有两份评测集——一份只用来测 Skill(rubric 锁死),一份只用来测 rubric(Skill 锁死)。我们目前是一份混着用,**这是飞轮"自评成功"陷阱的源头**
3. **"加严不放宽"规则会僵化**:某些 rubric 维度其实当初定得过严了(比如对输出格式要求得过死),"只加严"规则导致这些过严维度永远改不回来。需要一个"显式申请放宽"的流程,这个还不好做。
4. **多 Skill 共用 rubric 时的冲突**:`flow_adherence`、`information_completeness` 这类通用维度在 14 个 Skill 之间共享。一个 Skill 想改、其他 Skill 不想改时,要么分叉(维护成本爆炸)、要么妥协(没人满意)。这个治理问题目前完全靠人协调

核心结论

指标自动迭代的本质困难是**测量系统的自指**——你想用一个会自己变化的尺子,去衡量另一个会自己变化的物体。这个组合在任何工程系统里都是危险的。

飞轮真正的难点(一):线上问题怎么被"看见"

走到这里,有一个观察必须坦诚说出来:**搭起一个评测飞轮其实并不难**——auto-evaluation + pinchbench-eval 加起来不到 3 个月就跑通了,SKILL.md、rubric、queries.json 这些产物都在。但跑通之后我们才意识到,**飞轮转得动,不代表飞轮转得对**。

真正的核心难点有两个,这一节先讲第一个:**线上真实出现的问题,怎么进得了飞轮?**

当前的评测集是"我们想到的",不是"用户问出来的"

回顾 3.2 节:评测集是 auto-evaluation 基于"Skill 定义 + 真实业务数据"自动生成的。这个机制能保证评测集**贴近真实数据**(empId、商家名、任务列表都是真的),但**贴不近真实问法**——

- 真实小二会用各种意想不到的方式问问题:"那个谁家的高价怎么样了"、"昨天那批,搞完没"
- 真实小二会在中途打断、改主意、重新问
- 真实小二会同时问两个 Skill 才能回答的问题(跨 Skill 协作场景)

这些都不在我们设计的"典型/边界/追问/格式/触发词/真实 Case"6 类模板里。我们的 6 类是产研团队的视角,是"我们觉得应该测什么";真实流量是用户视角,是"用户实际在问什么"。**两者的交集是有限的**。

真实 Case 类目前是"伪真实"

3.2 节那张表里有"真实 Case"一类,占 5%。但坦白讲,这 5% 是**人工从钉钉群截图、转录、清洗后塞进去的**——本质上仍是人工策划,只是策划材料来自真实流量。代价是:

- **人工成本高**:一个真实 Case 从截图到能跑,至少 10–20 分钟(脱敏、补 empId、构造 reference_data)
- **频次低**:全靠人主动看群,看到了才录入,看不到就漏了
- **选择偏差**:能引起小二抱怨、被截图的,通常是显眼的失败;那些"不太对但小二自己改改也能用"的细微问题,基本进不来

结果是:**评测分数在涨,线上抱怨不一定在降**。这是飞轮转得动但转得不对的最典型表现。

理想状态:线上日志 → 异常检测 → Case 抽取 → 进入飞轮

我们想要的是一条自动化的"问题回流"链路:

```
IdeaWork 平台日志
    ↓ (1) 异常信号检测
异常对话片段
    ↓ (2) Case 抽取与脱敏
候选评测 Case
    ↓ (3) 人工 / 半自动评审
入库 queries.json
    ↓
进入 auto-evaluation 闭环
```

每一步都有具体的子问题,目前都没解好。

子问题 1:"异常"怎么定义?

线上日志没有"对/错"标签——用户没说"你答错了"的对话,可能是答对了,也可能是答错了用户自己改用别的方法。可候选的异常信号:

信号	含义	局限
用户连续追问同一意图 ≥3 轮	暗示前面没答好	也可能是用户在主动细化需求
用户说"不对/换一个/算了"	显式负反馈	出现频率低,不到 1%
会话超长(token 数 / 轮次 ≥ P95)	疑似陷入死循环	复杂任务也会超长,误报多
同一 empId 短时间内重启会话	暗示前一个会话不可用	有可能就是切了别的事情

我们当前的判断:**单一信号都不靠谱,需要组合 + 人工兜底**。但组合规则怎么写、阈值怎么定,还在摸索。

子问题 2:从对话流里"抽取"成 Case

线上对话不是评测 Case 的格式——它没有 `task_id`、没有 `category`、没有 `reference_data`。从一条原始对话变成一条可入库的 Case,需要:

- 切分:从一长串多轮对话里,切出"独立可重放的"一段
- 补元数据:推断这段对话的 category(典型/边界/追问)、关联哪个 Skill
- 构造金标:**这一步最难**——线上对话里没有"标准答案",得反过来根据 Agent 的实际输出 + Skill 定义,猜应该是什么样
- 防泄漏:reference_data 里不能直接抄 Agent 的输出,否则评测就变成"自评成功"

目前这一步基本靠人工。我们试过用 LLM 自动抽取,但抽出来的 Case 质量不稳定——LLM 经常**把 Agent 错的答案当成金标**(因为它读的就是 Agent 输出)。

子问题 3:隐私脱敏

线上日志里有真实商家名、员工工号、商品 ID,直接入库到 queries.json(进 Git)是合规风险。我们目前的做法是:

- 工号:替换为"测试号 236704"等几个固定 ID
- 商家名:保留(因为商家名本身是评测的关键参数,改了就跑不出真实场景)
- 商品 ID:保留(同上)

这个折中其实并不理想——商家名和商品 ID 在合规上也敏感。理想方案是建一套"评测专用的影子数据集",但这又会偏离"真实数据"的初衷。这是一个**没有完美解的工程取舍**。

当前阶段的选择

诚实地说,我们目前还没把"问题回流"做到自动化,采取的是务实的折中:

1. 行业小二的群消息**人工每周扫一次**,提取 3–5 条值得入库的 Case
2. IdeaWork 平台的"用户负反馈"按钮(当用户主动点踩时)被作为优先回流源
3. 大促期间专门拉一个"问题收集"群,鼓励小二随手截图

这些都是过渡方案。**真正的全自动飞轮,必须先解决"线上问题怎么自动进飞轮"**——否则飞轮再快,优化的也只是产研自己想到的那部分,线上真实问题永远滞后。这是 Q2 我们最优先的工程目标。

飞轮真正的难点(二):修改建议如何"改对地方"

假设问题被发现了——评测报告显示某个 Skill 在某些 Case 上分数低,接下来的问题是:**怎么改?改哪里?改完真的有效吗?**

这是飞轮的另一个核心难点,而且**比"发现问题"还要难**。

现状:LLM 给修改建议很容易,但 80% 改不对

auto-evaluation 的 Phase 4(指标蒸馏)和 Phase 5(Skill 蒸馏)会让 LLM 分析评测结果,给出修改建议。LLM 给建议这件事,**几乎是零成本**——你把评测报告、低分 Case、Skill 定义喂进去,它能立刻输出一份看起来很合理的修改方案。

但实际操作下来,我们的经验是:**这些建议有大约 80% 是不合格的**。具体表现:

- 改完之后下次评测分数确实涨了,但**只是因为评测集被同步改了**(LLM 把评测集和 Skill 一起改了),实际能力没提升
- 改完之后某个维度涨了,但**另一个维度跌了**(就是 4.3 节案例 3 的过度拟合)
- 改完之后所有 Case 都通过了,但**线上小二反馈反而变差**(因为评测集没覆盖那些场景)
- 改完之后**改了根本不该改的地方**(比如问题在脚本逻辑,LLM 改了 SKILL.md 文字)

为什么 LLM 给建议不准?根本原因是它**不知道也判断不了"问题的根因到底在哪"**。

根因归因:LLM 区分不了三种"低分"的真正原因

一个低分 Case,理论上可能是 4 种原因之一:

真实根因	应该改的地方	LLM 的常见误判
Skill 流程定义不清晰	SKILL.md	✅ 这个 LLM 一般能判对
Skill 脚本本身有 bug	scripts/*.py	❌ LLM 倾向改 SKILL.md
Rubric 评分标准过严	rubric_config.json	❌ LLM 倾向改 SKILL.md
评测集 reference_data 错了	queries.json	❌ LLM 倾向改 SKILL.md

LLM 的默认偏好是"改 SKILL.md"——因为 SKILL.md 是它最熟悉、最容易改的对象。但真正的问题可能在脚本、在 rubric、甚至在评测集本身。这就导致大量"改了等于没改"或"改了反而更糟"的情况。

举个真实例子:某个 Skill 的 `flow_adherence` 维度持续低分,LLM 给的建议是"在 SKILL.md 里加更多【强制】标记"。改完后下次评测确实涨了一点。但深入排查发现,**真正的根因是 scripts 里有个分支判断写反了**——Agent 调用脚本时本来就拿不到正确返回,无论 SKILL.md 怎么写都没用。LLM 看不到脚本层面的执行细节,只能在它能看到的 SKILL.md 上"修修补补"。

修改尺度:LLM 倾向"大改",而工程上需要"小改"

LLM 的另一个倾向是**修改尺度过大**——给建议时,经常一口气重写半个 SKILL.md。但工程上的良好实践应该是"**一次只改一个变量**":

- 一次改一处,下次评测能精确归因(分数涨/跌是因为这处改动)
- 一次改多处,即便分数涨了,你也不知道是哪个改动起了作用——下次想复用经验时无从下手

这其实和软件工程里"一个 commit 只做一件事"是同一个道理。但 LLM 没有这种工程直觉,它倾向于"既然要改,就一次性改完"。

我们目前的做法是**人工拆建议**:LLM 给一份 10 条修改建议,我们手动拆成 10 个 PR,每个 PR 跑一次评测验证。代价很高,但目前没找到更好的办法。

缺少"修改后验证"的强约束

第三个问题:Phase 5 改完 SKILL.md 之后,流程上**没有强制要求重新评测**——理论上应该改一次评一次,但实际经常因为"评测要等几十分钟"而被跳过,直接进 Phase 6 提 PR 上线。

更糟的是,即便重新评测,验证的也是**同一份评测集**——而那份评测集刚才被 LLM 一起改过了。这就是"自评成功"的陷阱:改 Skill 的同时改评测集,然后用改过的评测集证明 Skill 改对了。

我们当前的应对(都不完美)

- **Phase 4/5 强制人工 Review**:不是技术解,是流程兜底。代价是慢,而且依赖 Reviewer 的水平
- **Skill 改动 vs 评测集改动分两个 PR**:用 Git 流程把"被测对象"和"测试用例"的修改隔开,避免一锅炖
- **关键 Skill 留"冻结评测集"**:某些核心 Skill 的评测集打 tag 后就不再修改,任何 SKILL.md 改动都用冻结集回归。这相当于人工模拟"回归测试套件"的概念
- **改动后必须跑 N=3 次评测**:防止单次评测的随机性误导决策(对应 3.10 节讲过的 LLM Judge 漂移)

仍然没解的硬骨头

诚实列一下还没解的:

1. **如何让 LLM 知道"应该改哪一层"**:需要把 Skill / 脚本 / Rubric / 评测集这 4 层的边界,以可被 LLM 理解的方式喂给它。目前还没找到好的表达方式
2. **如何自动检测"自评成功"**:Skill 和评测集同时改时,如何识别这是真改进还是循环自证。可能需要引入"对照评测集"的概念(永远不变的那一份)
3. **如何把"小步快跑"工程化**:把 LLM 给的 N 条建议自动拆成 N 个独立验证流程。这件事本身需要不少基础设施

5.5 小结:出口的问题最终会回到一个共同根源——log

5.5 讲到这里,所有"应对"其实都依赖一个隐含前提:**我们能追溯过去发生过什么**。可一旦认真去翻——AI 上次给了什么建议、Reviewer 为什么拒绝、改完之后验证有没有跑——会发现这些**根本没有被持久化**。这是一个比"LLM 改不对"更底层的问题,5.6 节专门讲。

飞轮真正的难点(三):没有"修改痕迹"的飞轮等于没有飞轮

5.4 讲了飞轮的入口、5.5 讲了出口,但**飞轮真正"转起来"还有第三个前提:每一次转动都被记录下来**。否则下一圈不知道上一圈发生了什么,所谓的"持续优化"其实是每次都从零开始。

这一节坦白讲——**这件事我们目前做得最差**。

现状盘点:模板齐全,实例为零

auto-evaluation Skill 在设计时就考虑到了"修改要留痕",`templates/` 目录下定义了完整的产物模板:`skill_fix_suggestion.md`、`metric_suggestion.md`、`annotated_results.json`、`iteration_log.md`。**但跑了几个月之后,我们去翻全仓——这四类模板对应的实例文件,一个都没有**。

具体到三类操作:

操作类别	模板	实例落盘	与 Case 关联	版本化	现状
Skill 修改	有	**无**	无	仅 git commit	改了什么靠 `git diff` 看,改的原因靠 commit message 猜
人工标注	有	**无**(目录为空)	无	无	标注员靠对话临时承载,提交一次就消失
指标修改	有	**无**	无	无	rubric 文件单一可变,改动只在 git diff 里

`pinchbench-suite/outputs/<run>/human_scores/` **目录在所有 run 里都存在,但全部为空**——这个目录是我们设计时给人工标注准备的"信箱",但没人往里寄信。

为什么这个缺口比看上去严重

没有 log,**Phase 4/5 的 LLM 蒸馏会持续退化**——每次都是"基于这次评测看,应该改什么",而不是"基于过去 10 次评测的修改历史看,什么样的改动真正有效"。LLM 学不到自己历史决策的正反馈,**飞轮就只是空转,没有累积**。

具体说,缺 log 直接导致以下三件事做不到:

1. **回答不了"上次为什么这么改"**——3 个月后看 SKILL.md 某段约束,不知道当初是因为哪个 Case 加进去的;想删的时候,**也不知道删了会不会让那个 Case 重新出问题**
2. **Phase 4/5 没有训练数据**——AI 给的建议被采纳还是被拒,目前没有任何记录,所以 LLM 永远不知道自己上次的建议质量如何,**每次都是从零猜**
3. **跨人交接成本巨高**——一个 Skill 换负责人,新人完全没法继承"哪些改动是有意为之、哪些是历史包袱",只能重新踩一遍坑

我们的设计:三类 log + 一类 AI 建议追踪

基于诊断,我们设计了一套以 **Skill 为聚合单元**的 log 体系,每个 Skill 下统一四个 log 流,**全部 JSONL 格式,按月分文件**:

```
skill-data/skills/<skill_name>/
├── queries.json
├── rubric_config.json
└── _logs/
    ├── annotation/          # 人工标注事件
    │   ├── 2026-05.jsonl
    │   └── 2026-04.jsonl
    ├── rubric_change/       # 指标修改事件
    │   └── 2026-05.jsonl
    ├── skill_change/        # Skill 修改事件
    │   └── 2026-05.jsonl
    └── ai_suggestion/       # AI 建议及其结局
        └── 2026-05.jsonl
```

为什么按 Skill 而不是按 run 聚合:run 是临时的,Skill 才是稳定的"问题域"。Phase 4/5 蒸馏时要扫的是"这个 Skill 历史上的全部修改",按 Skill 聚合让查询和归档边界都清晰。这一点和工作台整体"以 Skill 为一等公民"的设计一致。

为什么按月分文件:增长边界天然存在(单 Skill 单月文件再大也有限),不需要"达到 N 条才归档"这种触发逻辑——**没有归档脚本就没有归档 bug**。

四类 log 的字段约定(简版示例)

`annotation/2026-05.jsonl`(人工标注):

```
{
  "annotation_id": "ann_20260508_001",
  "timestamp": "2026-05-08T14:32:11+08:00",
  "annotator": "236704",
  "task_id": "task_seller_broadcast_typical_003",
  "rubric_version": "v3",
  "verdict": "false_positive",
  "reason": "LLM 误判:Agent 跳过了'查岗位权限'步骤,但 LLM 没识别出来",
  "suggest_action": "rubric.flow_adherence 加严:必须显式列出权限校验调用"
}
```

`rubric_change/2026-05.jsonl` (指标修改):

```jsonl
{
  "change_id": "rub_20260508_001",
  "rubric_file": "flow_adherence.md",
  "from_version": "v2", "to_version": "v3",
  "trigger_annotations": ["ann_20260508_001", "ann_20260508_005"],
  "regression_run": "all-skill-eval-20260508-regression",
  "regression_result": "78/80 排序稳定,2 翻转(已确认合理)",
  "approved_by": "236704",
  "ai_suggestion_id": "phase4_sug_20260508_001"
}
```

`skill_change/2026-05.jsonl` (Skill 修改):

```jsonl
{
  "change_id": "skl_20260508_002",
  "files_changed": ["scripts/build_message.py"],
  "trigger_run": "all-skill-eval-20260507",
  "trigger_low_score_tasks": ["task_seller_broadcast_typical_003"],
  "ai_suggestion_id": "phase5_sug_20260508_002",
  "human_modification": "AI 建议改 SKILL.md,人工排查后定位根因在 scripts,改
  "verify_run": "seller-broadcast-eval-20260508-verify",
  "verify_result": "原 8 个低分 Case 全部通过,无新增低分",
  "git_commit": "5f18cb2"
}
```

`ai_suggestion/2026-05.jsonl` (AI 建议追踪):

```jsonl
{
  "suggestion_id": "phase4_sug_20260508_001",
  "phase": "phase4",
  "input_annotations": ["ann_20260508_001", "ann_20260508_005"],
  "suggested_change": "flow_adherence 加严:必须显式列出权限校验调用",
  "outcome": "adopted",
  "outcome_reference": "rub_20260508_001"
}
```

关键设计:四类 log 通过 ID 互引(`trigger_annotations` / `ai_suggestion_id` / `outcome_reference`),构成完整的因果图——**从一条标注,可以追到它如何影响指标、又如何驱动 Skill 改动、最终验证有没有效果**。这条因果链就是 Phase 4/5 LLM 未来要学的训练数据。

写入时机:log 不记"做了什么",log 只记"决定了什么"

这是 log 体系最关键的设计判断。开发者调 SKILL.md 改了又改,标注员对一条 Case 反复看反复评——**这些都是"过程",不是"事件"**。只有当一个动作变得"无法回头",log 才落盘:

Log	写入时机(决策点)	为什么不更早
`annotation`	标注员点击"确认提交"	看了/改了但没提交,不算决策
`rubric_change`	rubric PR 被 approve	AI 出建议、人在改文件,都还是探索
`skill_change`	Skill PR merge 到主干	调试中间状态是噪声,不是事实
`ai_suggestion`	Phase 4/5 LLM 输出建议时	LLM 思考过程不是事件,"输出"才是

这样过滤之后,一次完整的 Skill 优化(评测 → 标注 → 指标改 → Skill 改 → 验证)产生的 log 大约在 10–15 条之间,**全是决策事件,没有调试噪声**。Phase 4/5 后续从这十几条里能学到完整的因果链——**这才是健康的密度**。

唯一例外是 AI 建议:即使没被采纳也要记。被拒的建议是 LLM 的负样本,价值和正样本一样大——没有这部分,LLM 永远学不到"我哪类建议会被否"。

消费侧:JSON 是 source of truth,Markdown 是 view

log 全部 JSONL,但人不会去读原始 JSONL——通过 `scripts/render_log.py` 现场渲染成 Markdown:

```
# 单 Skill 月度报告
python scripts/render_log.py --skill seller-broadcast-sender --month 20
# 单条 log 的因果链(标注 → 指标改 → Skill 改 → 验证)
python scripts/render_log.py --trace ann_20260508_001
# 跨 Skill 季度汇总
python scripts/render_log.py --quarter 2026Q2
```

这样既保证了**结构统一**(对程序友好,跨 Skill 聚合不用做格式转换),也保证了**人想看就能看**(渲染脚本一行命令的成本)。**JSON 是事实层,Markdown 是阅读层,职责分离**——这一点是讨论中我们最早就否决了"按用途混用 JSON 和 Markdown"的原因:混格式长期一定会腐烂。

当前阶段(都不完美)

1. **设计已敲定,落地刚启动**:四类 log 的字段定义和 `render_log.py` 已经在写,Phase 0.5(新 Skill 初始化)流程会自动建 `_logs/` 目录
2. **存量 Skill 没有历史 log**:已经跑过的几次评测产物(4/22 baseline、5/3、5/4、5/7)只有报告没有标注 log——这部分**永远补不回来了**,这是缺口的代价
3. **写入门槛靠流程兜底**:目前没法从代码层强制"PR merge 必须写 log",还是靠 SKILL.md 里写硬约束 + Reviewer 检查。这是临时方案,理想是接 Git hooks

仍然没解的硬骨头

1. **如何让历史 log 真正驱动 Phase 4/5**:有了 log 不等于 LLM 会用 log。Phase 4/5 当前的 prompt 设计还没考虑"先读历史 log、识别模式、再给建议"。这件事需要 prompt 工程上的重新设计
2. **跨 Skill 共享 rubric 的 log 归属**:`flow_adherence` 被多个 Skill 共用,改动可能由 Skill A 触发但影响 Skill B/C。我们目前的策略是 log 写到触发的 Skill 下,影响的 Skill 通过 ID 引用。**但跨 Skill 查询体验目前还很差**
3. **log 的"信噪比"治理**:即使做了"只记决策"的过滤,半年后 annotation log 也可能积累几千条。怎么自动识别"高价值标注"(比如反复出现同类误判的 Case)、把噪声标注降权,是后续要解的问题

核心结论

回头看 5.4、5.5、5.6 这三节,它们其实是评测飞轮的**入口、出口、记忆**——三件事都没有任何一件是"工程上很复杂",但**每一件都是"做了才知道难"**。

- **入口(5.4)**:线上真实问题进不来,飞轮就是"自娱自乐"
- **出口(5.5)**:修改建议改不到点子上,飞轮就是"原地空转"
- **记忆(5.6)**:每次转动不被记录,飞轮就是"每圈都从零开始"

Phase 0~3(评测集生成、执行、报告)我们做得差不多了——这是飞轮的"中段",也是最容易的部分;Phase 4~5(指标 / Skill 蒸馏)还在依赖大量人工兜底,卡在 LLM 的根因判断能力上;**而真正的入口(线上问题回流)和记忆(log 体系)还基本是空白**。这是接下来一年最值得投入的方向,远比"把指标自动迭代做掉"重要。



把话说到底:**飞轮造起来不难,难的是让飞轮真的转出价值**。这是我们目前最深的体会,也是这篇文章想要传达的核心经验。

06 未来演进方向

下一阶段的目标:不是"去掉人工",是"让人工更值钱"

走完前五章的复盘后,我们对"全自动"这个词的理解和半年前完全不一样了。**前面的结论都指向同一件事——目前阶段强行追求"无人值守"是危险的**:指标自动迭代是测量系统的自指、LLM 改建议 80% 改不对、决策 log 还基本是空白。在这三个底层问题没解之前,把人工从流程里拿掉只会让飞轮更快地"自评成功"。

所以下一阶段的目标不是"全自动",而是:**把人工从机械确认环节解放出来,集中在真正需要判断的地方**。具体三条主线:

主线一:入口——把线上问题接进飞轮(对应 5.4)

这是我们目前最优先的工程目标。目前的"伪真实 Case"靠人工每周扫群,频次低、选择偏差大。下一阶段要做的:

- **异常信号采集**:基于"用户连续追问"、"显式负反馈"、"会话超长"等组合信号,从 IdeaWork 平台日志里捞出候选异常对话
- **半自动 Case 抽取**:LLM 做切分 + 元数据补全 + 脱敏的初稿,人工只做"采纳/打回"的二选一判断
- **目标**:把"产研团队想象中的场景"和"线上真实场景"的覆盖差,从目前的"基本不知道"变成"可量化"

不是"全自动入库"——5.4 节列的三个子问题(异常定义、金标构造、隐私脱敏)目前都还没好的解,所以人工把关短期不会取消。

主线二:记忆——log 体系真正落地(对应 5.6)

5.6 节设计的四类 log(annotation / rubric_change / skill_change / ai_suggestion)目前实例还是零。下一阶段要做的:

- **写入门槛工程化**:从"靠 SKILL.md 写硬约束"升级到"Git hooks 强制校验"——PR merge 没附 log 就 block
- **render_log.py 上线**:JSON 是事实层、Markdown 是阅读层,这条职责分离要在工具上落地,不能只是文档约定
- **存量补录**:已经跑过的几次评测产物没有 log,这部分永远补不回来——但**新评测必须开始有 log**,这是硬底线

log 体系一旦跑起来,**Phase 4/5 才有真正的训练数据**——LLM 才能从"基于这次评测看应该改什么"进化到"基于过去 10 次的修改历史看什么样的改动真正有效"。这是后面所有"更聪明的自动化"的前提。

主线三:出口——修改建议精准归因(对应 5.5)

5.5 节指出 LLM 给的建议 80% 不合格,根因是它判断不了问题真正在哪一层。下一阶段不指望靠模型升级解决——而是用工程手段把"应该改哪一层"显性化:

- **分层评测集**:逐步建立"对照评测集"(永远不变的那一份)和"演进评测集"(随场景增长)的分离,识别"自评成功"
- **建议拆分流水线**:LLM 给一份 N 条修改建议,自动拆成 N 个独立 PR + 独立验证,代替目前手工拆的方式
- **改动后强制回归**:Phase 5 改完到 Phase 6 之间加硬卡口——必须跑过冻结评测集回归才能提 PR

仍然要承认:**这三条主线没有一条能在一个季度内做完**——它们都是"做一年才看得出效果"的基础设施投入。我们能给的承诺是优先级,不是交付时间。

为什么"指标自动迭代"反而排到最后

之前我们以为"指标自动迭代"是离全自动最近的一步——评分标准最容易让 LLM 改。实际跑下来反而是最难的:它是测量系统的自指,没有外部锚点能告诉你改对了还是改坏了。所以下一阶段它不是被推进,而是被**主动延后**——等入口和记忆做出来,有了"独立 ground truth + 历史决策训练数据"两个前提,再回头碰它才有意义。

一个具体的借鉴:autoresearch 给出的最小骨架

Karpathy 在 2026 年 3 月放出的 autoresearch 是这个方向上最干净的参考。它让 AI agent 自主修改 `train.py`、按固定 5 分钟预算跑实验、val_bpb 涨了就 keep 不涨就 `git reset`,一晚上跑一百次实验。

看完这个框架后我们问的第一个问题是:"能直接搬过来做 Phase 4/5 蒸馏吗?"答案是**不能直接搬**——autoresearch 的自动化靠四个假设:度量是数学真理、单一标量、失败无副作用、可

以 NEVER STOP。这四条我们一条都不成立:LLM Judge 自身就有 5~10% 漂移、pass_rate 是多维度均分(4.3 案例 3 证明了能被刷)、改动会被同步到云端 IdeaWork、无限循环会把假阳性变成结构性污染。

但它的**循环骨架**值得借鉴——我们据此做了一份叫 SkillResearch 的方案,把 autoresearch 的四个设计保留、四个假设全部换成适合语义类指标的工程化版本:

设计点	autoresearch	SkillResearch
白盒 / 黑盒分离	train.py 可改,prepare.py + evaluate_bpb 禁改	SKILL.md 可改,rubric + queries + pinchbench-eval 禁改
度量	单一标量 val_bpb	pass_rate 主指标 + 5 维度钳制(任一跌 0.05 强制 discard,对应 4.3 案例 3 的"反向 Case")
重复策略	单次(loss 确定)	N=3 均值(降 LLM Judge 漂移)
分支策略	直接 keep 在工作分支	严禁触碰 main,所有实验在 `skillresearch/*` sandbox,keep 只是"候选",人工 review 才能 merge
停止条件	NEVER STOP	4 个主动停止条件(连续 5 次 discard / 累计 3 次 keep / 噪声 > 0.05 / 超时)
决策记忆	results.tsv 5 列	results.tsv 8 列,多出 `hypothesis` + `prior_ref` + `outcome_log_id` ——最后一列接入 5.6 节的 ai_suggestion log,让 agent 能看到"自己上次类似尝试的成败"

这个方案和三条主线的关系:SkillResearch 不是第四条主线,**是"记忆主线做到一定程度后、出口主线落地的具体形态"**。因为它最核心的那列 `outcome_log_id` ——"这次实验最终是否被采纳、在下一次评测里表现如何"——数据源正是 5.6 节设计的四类 log。log 体系不落地,SkillResearch 退化成 autoresearch 的拙劣模仿;log 体系落地之后,SkillResearch 是 Phase 4/5 走向真正自动化的骨架。

我们在 `skillresearch-prototype/` 下做了最小可行原型:`run_n3.sh`(N=3 wrapper + sandbox 安全栓)、`aggregate.py`(聚合 + 钳制判定)、`skillresearch.md`(agent 指令文件)。第一次 dry-run 跑通了真实 0508 评测数据的聚合逻辑,基线 pass_rate=0.8307,维度跌 0.07 能正确触发 discard 并指明哪个维度。但这只是工程骨架通了——**它真正的价值,要等三条主线里的"记忆"跑出第一批 ai_suggestion log 之后,才会显现**。这个时间差本身就是诚实的一部分:**工程骨架可以两周搭出来,但让这个骨架真正跑起来的燃料(log 因果链),至少要一年**。

取舍的诚实

我们没法给出 "Q2 做 A、Q3 做 B、Q4 做 C" 那种漂亮的甘特图。原因很简单:这三件事的难度我们自己也没把握,**任何一个 quarter 计划写出来都是猜的**。我们能做的是把优先级钉死(入口 > 记忆 > 出口 > 指标自动迭代),按这个顺序投资源,不被"先做容易出 demo 的事"带偏。这是从前五章的复盘里学到的最贵的一课。

07 总结

高价率运营 AI 工作台采用"约定驱动 + AI 编排"的架构设计,实现了产研闭环的自动化。核心做法在于:

1. **约定驱动架构**:通过目录结构固化开发规范,让规范成为默认选项,借鉴现代框架的"Convention over Configuration"哲学
2. **AI 智能编排**:通过自然语言驱动全流程自动化,从需求到发布无缝流转,实现"规范即代码、执行即编排、优化即闭环"
3. **双引擎评测体系**:auto-evaluation(评测大脑) + pinchbench-eval(执行引擎)分工协同,评测从一次性快照变为持续优化引擎
4. **真实数据驱动**:评测集基于真实业务数据生成,reference_data 采用"结构化字段 + 自然语言描述"的混合金标
5. **人机协同最佳实践**:AI 负责执行,人工负责决策——尤其是 Phase 4/5 的指标和 Skill 修订必须人工确认才能落盘

这套体系已经在实践中证明可行:Skill 评测通过率与问题识别率持续提升,人工标注占比明显下降,可量化、可复跑、可对比的评测产物已经成为团队工程文化的一部分。

但更想坦诚分享的,是飞轮跑通之后才暴露出来的三个真正难点(详见 5.4 / 5.5 / 5.6):

- **难点一(入口):线上问题怎么进得了飞轮**——评测集再丰富,如果只是"我们想到的场景",就永远滞后于"用户实际问的场景"。这是飞轮的入口问题,目前还基本是空白
- **难点二(出口):修改建议如何真正改对地方**——LLM 给修改建议很容易,但约 80% 的建议要么没解决问题、要么按下葫芦浮起瓢。根因是 LLM 难以判断问题真正出在哪一层(Skill/脚本/Rubric/评测集)。这是飞轮的出口问题,目前靠人工兜底
- **难点三(记忆):每一次转动如何被记录下来**——人工标注、Skill 修改、指标修改、AI 建议是否被采纳,这些"决策事件"目前都没有持久化。模板齐全,实例为零。没有 log,Phase 4/5 的 LLM 蒸馏永远学不到自己历史决策的成败,飞轮就只是空转,没有累积

把话说到底:**搭一个评测飞轮不难,让飞轮转出真实价值才难**。下一阶段最重要的工程目标,不是把"指标自动迭代"做到 100%,而是先打通**入口**(线上问题回流)、**出口**(修改建议精准归因)、**记忆**(决策 log 体系)这三条路。否则飞轮再快,转的也只是产研团队自己想象的世界。

这是一篇关于评测的实践记录,也是一份关于"评测做完了之后才发现真正的难题在哪儿"的诚实报告。希望这些经验对正在搭建 LLM Agent 评测体系的团队有所参考。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 2342

留言

写留言

还没有留言，来写首评